

STUDENT TEMPLATE

Smart Coaster — Hydration Tracker

Advanced IoT Systems

KLE Technological University • ECE Department

SENSE

PROCESS

COMMUNICATE

POWER

ACTUATE

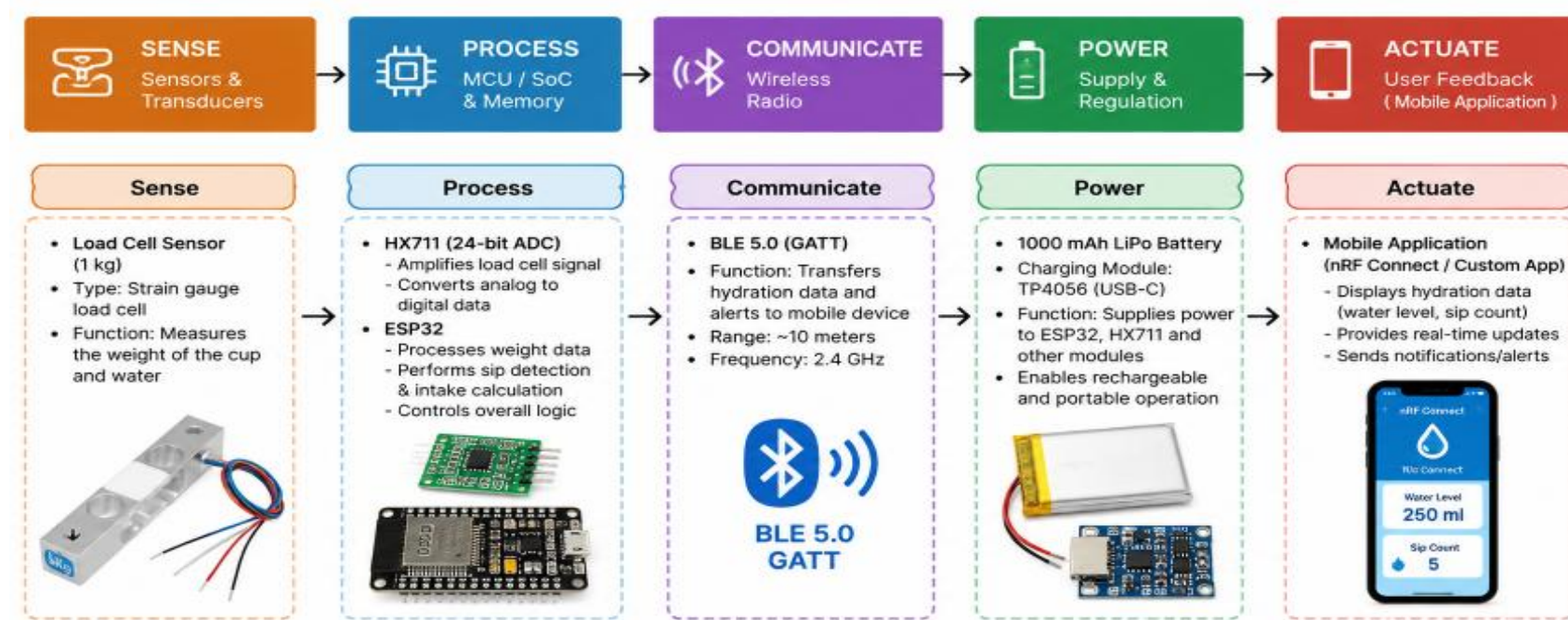
PROBLEM STATEMENT:

Most people fail to drink adequate water daily, leading to fatigue, headaches, and reduced cognitive performance. Existing hydration trackers require manual logging or smart bottles that are expensive and bottle-specific. No passive tracking solution exists for any cup or glass.



BLOCK DIAGRAM:

Map the product to the five IoT subsystems



Sense — Sensors & Transducers

Identify all sensing elements and their interfaces

1) Load Cell Sensor(5 KG)

Why?

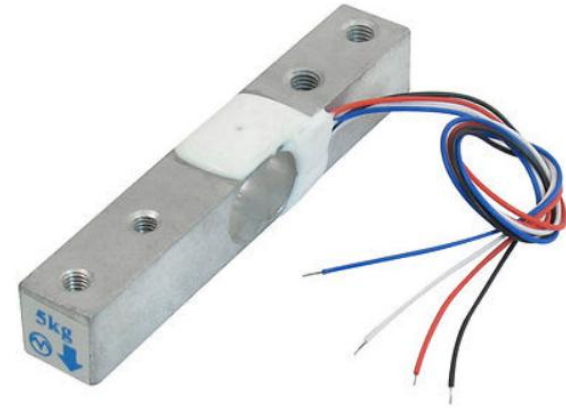
- Converts weight into electrical signal
- Works with any container (no fixed container needed)
- Provides continuous weight measurement
- Detects very small changes

Type : Load Cell / Weight Sensor

Capacity : 5 kg

Function: Measures the weight of the cup and water

Supply Voltage : 5V (via HX711)



Process — MCU / SoC & Memory

Identify the processing core, memory, and peripherals

2) HX711(24 Bit ADC)

Why?

- Amplifies very small signal from load cell
- Converts analog signal → digital data
- Enables microcontroller to read weight values

Type: HX711 Load Cell Amplifier + ADC

Interface: DT/SCK

4) ESP 32

Why?

- Processes weight data from HX711
- Performs sip detection & intake calculation
- Supports BLE communication
- Controls display, buzzer, and timer functions

Type :Microcontroller (ESP32 SoC)

Core :Dual-core processor

Function :Reads sensor data, processes logic, controls output devices, and manages communication

Interfaces :GPIO, I2C (OLED), SPI, UART, BLE



Communicate — Wireless & Connectivity

Identify wireless protocols, RF components, and antenna design

5) BLE 5.0 GATT

Why?

- Enables wireless data transfer
- Sends hydration data to mobile app
- Supports low power operation
- Provides real-time notifications

Type : Wireless Communication (BLE 5.0)

Function : Transfers intake data and alerts between ESP32 and mobile device

Range : ~10 meters

Frequency : 2.4 GHz

Power — Supply & Regulation

Trace the complete power path from input to every rail

6) 1000mAh Lipo+TP4056 USB-C

Why?

- Provides portable power supply
- Supports rechargeable battery operation
- Enables USB-C charging
- Ensures continuous system operation

Type : LiPo Battery + Charging Module

Battery Capacity : 1000 mAh

Function : Supplies power to ESP32, HX711

Charging Module : TP4056 (USB-C input)



Actuate — Outputs & Feedback

Identify all output and actuation elements

6) User Feedback through BLE

Type: Mobile Device Feedback via BLE

Interface: Bluetooth Low Energy (BLE)

Application: nRF Connect

Function:

- Receives hydration data from the ESP32.
- Displays current weight and water-level percentage.
- Displays the sip count received from the ESP32.
- Allows the user to send commands such as TARE and RESET.
- Provides real-time feedback without requiring a dedicated display on the coaster.

Why?

- Provides wireless feedback to the user.
- Eliminates the need for a dedicated OLED display.
- Allows the user to monitor hydration data remotely.
- Supports two-way communication between the ESP32 and mobile device.

Bill Of Material (BOM):

S.No	Components	Specification	Quantity	Price
1	ESP32 Dev Board	3.3V, Dual-core, BLE + WiFi	1	₹350 – ₹500
2	Load cell	5kg, Strain Gauge Type	1	₹150 – ₹300
3	HX711 Module	24-bit ADC, Gain 128/64	1	₹80– ₹150
4	LiPo Battery	3.7V, 1000mAh	1	₹150– ₹300
5	TP4056 Module	LiPo charger, Type-C/Micro USB	1	₹50– ₹100
6	Resistors	220Ω (for LED)	2-3	₹1
7	Connecting wires	Jumper wires	1 set	₹50– ₹100
8	USB cable	Type-C / Micro USB	1	₹50– ₹150

Power Budget

Components	Voltage(V)	Current (mA)	Power(mW)
ESP32 (Active + BLE)	3.3	80-150	264-495
ESP32 (Avg)	3.3	80	264
ESP32 (Sleep)	3.3	0.01-0.15	0.03-05
HX711	3.3	1.5	5
Load Cell	3.3	1	3.3

load cell mounting plan

- Load cell mounted between bottom base and top coaster plate
- Base side fixed rigidly
- Top side connected to cup platform
- Cup load transferred vertically through sensor
- Sensor positioned near center for balanced force distribution
- Mechanical structure designed to reduce tilt and vibration

Schematic walkthrough: HX711 interface

HX711 driver working

- The HX711 is interfaced with ESP32 using DT and SCK pins.
- The microcontroller reads 24-bit digital data from HX711.
- We implemented a timer-based driver instead of delay to ensure efficient and stable data acquisition.
- Averaging is used to reduce noise and improve accuracy.

load cell calibrated (gram accuracy)

- Known weight placed on scale
- Raw value recorded
- Scale factor calculated
- Offset set for zero reference

Tested with known weights

- Achieved gram-level accuracy
- Stable readings obtained
- Noise reduced using averaging

Formulae used in calculating

- $\text{Weight} = (\text{Raw} - \text{Offset}) / \text{Scale Factor}$
- $\text{Scale Factor} = (\text{OFFSET} - \text{Raw}) / \text{Known Weight}$

Schematic walkthrough: Analog ground strategy

Why Ground Separation is Needed?

- Load cell output is a very small millivolt signal
- HX711 reads low-level analog signals with high sensitivity
- ESP32-S3 generates switching noise from CPU, Wi-Fi, clocks, GPIO activity
- Poor grounding can cause unstable weight readings

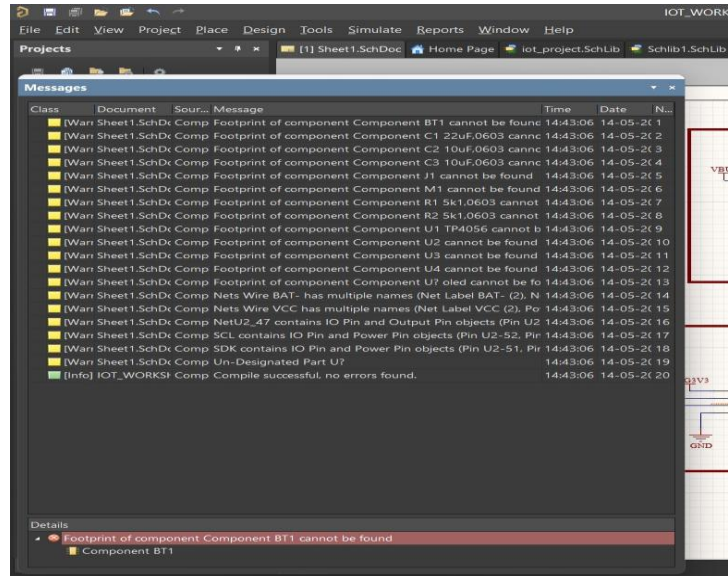
Design Strategy Used

The load cell generates very small analog signals, which are highly sensitive to noise. To ensure accuracy, we followed an analog separation strategy by placing the HX711 close to the sensor, using short connections, and maintaining proper grounding. This reduces noise and improves measurement stability.”

Title	Smart Coaster Hydration Tracker		
Size	Number	Revision	
A4			
Date	5-15-2026	Sheet of	
File	C:\Users\jarnat\coaster hydration\Sch	Drive By	

ERC report

The schematic design was verified using Electrical Rule Check (ERC) in Altium Designer. All electrical connections, power nets, and signal paths were validated successfully. The design meets all required electrical constraints and is ready for implementation.



Software Architecture

1. Drivers

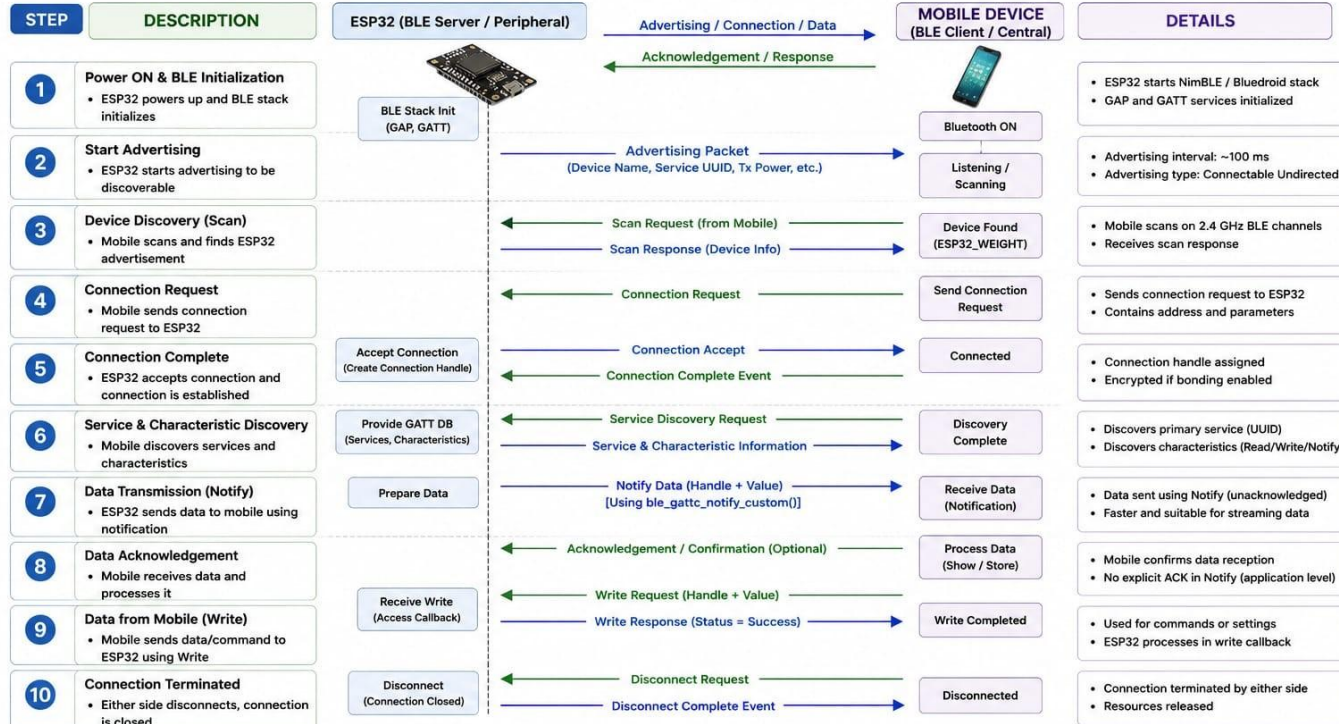
ESP32 ↔ HX711 COMMUNICATION (DT & SCK)

STEP	ESP32 (MASTER)	HX711 (SLAVE)
1	Power ON & Initialization	Power ON, Initialize HX711 Interface (DT, SCK)
2	Check DT Pin (Data Ready)	Sends DT Status
3	Wait for Data Ready	Pulls DT LOW When Data is Ready
4	Start Data Read	Ready to Transmit Data
5	ESP32 Sends Clock Pulse (1)	Receives Clock Pulse
6	HX711 Sends 1 Bit on DT (1)	Outputs 1st Data Bit on DT
7	Repeat for 24 Bits	Sends 24 Bits (MSB to LSB) on DT Pin
8	ESP32 Sends 25th Clock Pulse	Receives 25th Clock Pulse (Gain = 128), Prepare Next Data
9	Data Received in ESP32	Conversion Cycle Completed
10	HX711 Starts Next Conversion	Starts Next Conversion
11	Next Data Ready	Next Data Ready

LEGEND:  ESP32 → HX711  HX711 → ESP32

2.Communication

ESP32 ↔ BLUETOOTH (BLE) COMMUNICATION



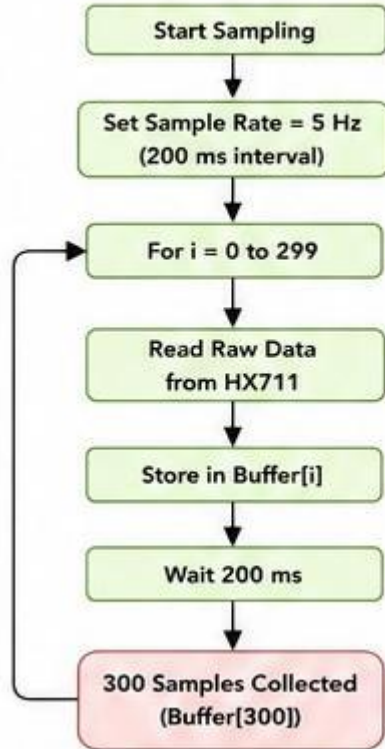
SUMMARY

- ESP32 acts as BLE Server (Peripheral) and Mobile acts as BLE Client (Central).
- Connection is established after successful advertising and scanning.
- After connection, data can be sent from ESP32 (Notify) and received by Mobile.
- Mobile can also send commands/data to ESP32 (Write).
- Either side can disconnect and connection is properly closed.

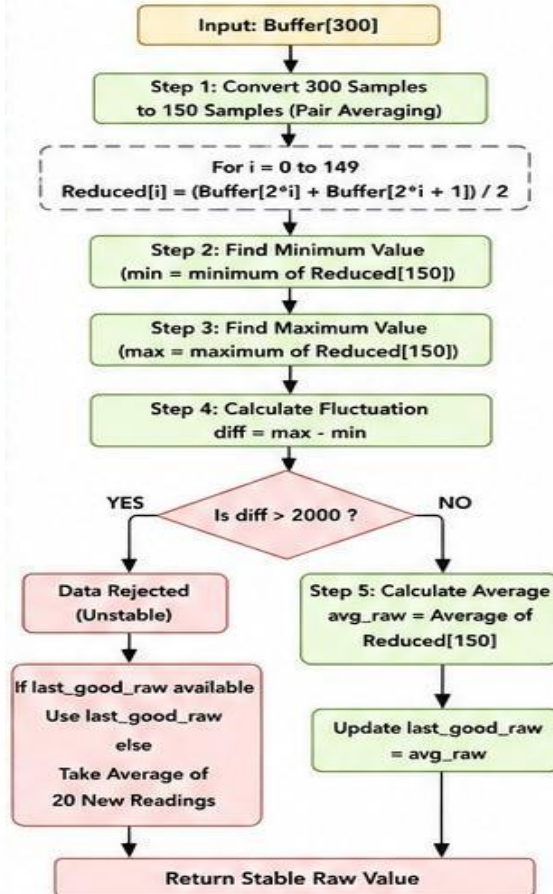


3.Algorithm

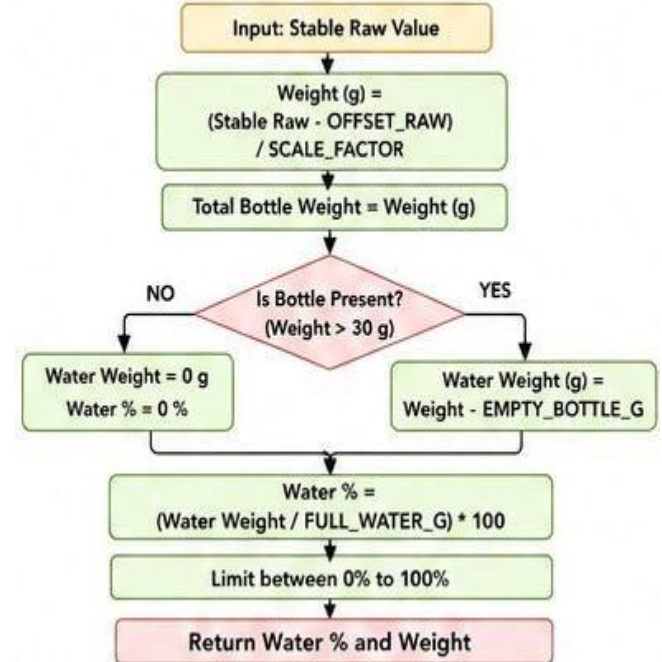
SUB PROCESS 1 COLLECT 300 SAMPLES



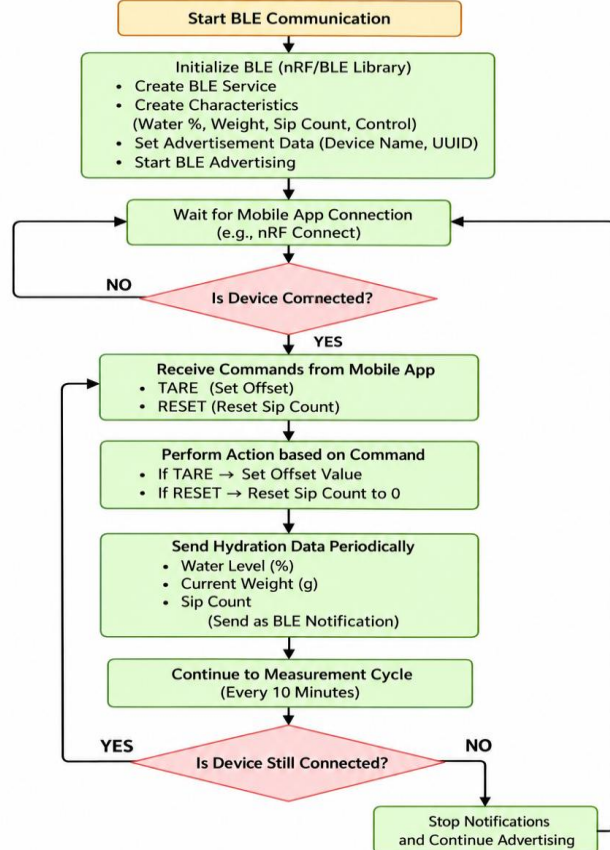
SUB PROCESS 2 BUFFER PROCESSING ALGORITHM



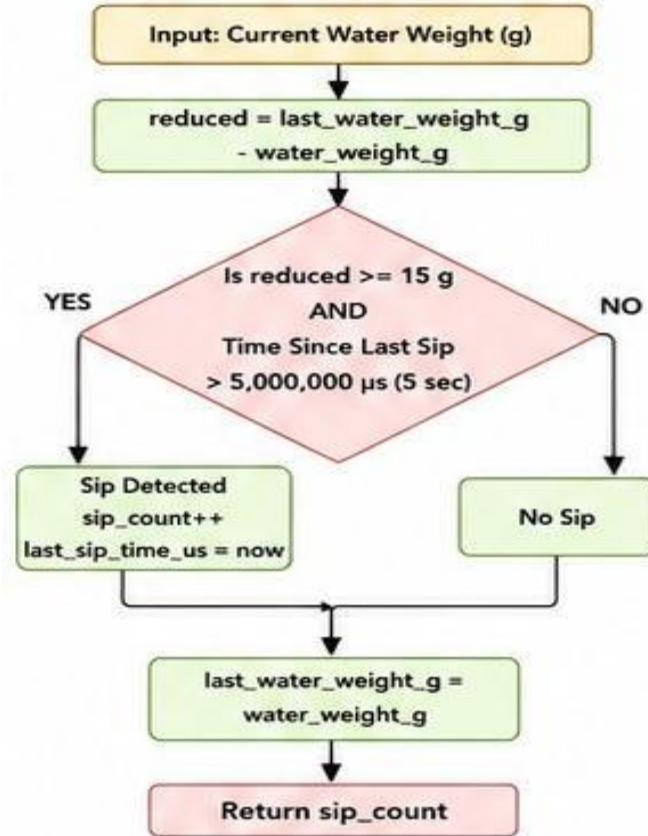
SUB PROCESS 3 WEIGHT & WATER LEVEL CALCULATION



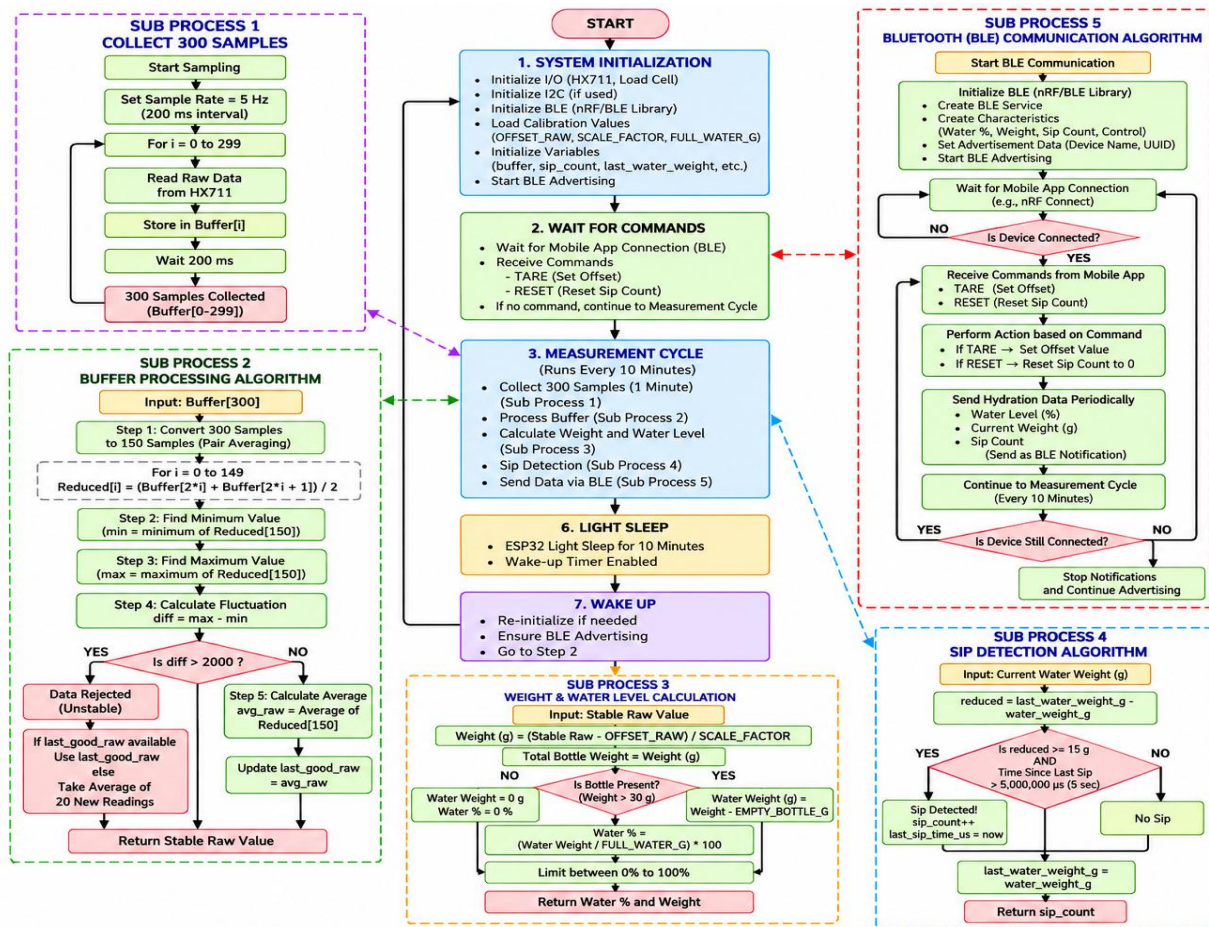
SUB PROCESS 4 BLUETOOTH (BLE) COMMUNICATION ALGORITHM



SUB PROCESS 5 SIP DETECTION ALGORITHM



Detailed Architecture



Result Screenshots

The screenshot shows an IDE with a C file named `hello_world_main.c`. The code defines a `main` function that calls `app_main(void)`, which in turn calls `size_of(keepalive)`. The `size_of` function returns a string containing sensor data: `"%s %dmg %dmg S:%d", time_str, total_weight_g, level_percent, sip_count;`. Below the code, the terminal window displays the following logs:

```
I (618927) NimBLE: att_handle=16
Keepalive Notify rc=0
I (648927) NimBLE: GATT procedure initiated: notify;
I (648927) NimBLE: att_handle=16
Keepalive Notify rc=0
10 minutes completed. Taking next reading...
collecting 300 samples...
Sample collection completed
Min = 630678
Max = 633379
Fluctuation = 2701
UNSTABLE DATA - REJECTED
06:15 AM 438g 85% S:0
I (732867) NimBLE: GATT procedure initiated: notify;
I (732867) NimBLE: att_handle=16
Waiting 10 minutes...
I (732867) NimBLE: GATT procedure initiated: notify;
I (732867) NimBLE: att_handle=16
Keepalive Notify rc=0
I (762877) NimBLE: GATT procedure initiated: notify;
I (762877) NimBLE: att_handle=16
Keepalive Notify rc=0
I (792877) NimBLE: GATT procedure initiated: notify;
```

The screenshot shows a mobile application interface with a blue header bar containing the time '6:17' and various status icons. Below the header, there is a 'Devices' section with a 'CONNECT' button. A table lists device information:

BONDED	ADVERTISER	ESP32_WEIGHT
		1C:DB:D4:44:50:82

Below the table, there is a 'DISCONNECT...' button and a 'CLIENT' tab. The logs show the following sequence of events:

- 18:14:35.283 Notification received from 0000fff1-0000-1000-8000-00805f9b34fb, value: (0x) 30-36-3A-31-34-20-41-4D-20-33-38-37-67-20-37-35-25-20-53-3A, "06:14 AM 387g 75% S:"
- 18:14:35.283 "(0x) 30-36-3A-31-34-20-41-4D-20-33-38-37-67-20-37-35-25-20-53-3A, "06:14 AM 387g 75% S:" received
- 18:16:07.221 Notification received from 0000fff1-0000-1000-8000-00805f9b34fb, value: (0x) 30-36-3A-31-35-20-41-4D-20-34-33-38-67-20-38-35-25-20-53-3A, "06:15 AM 438g 85% S:"
- 18:16:07.221 "(0x) 30-36-3A-31-35-20-41-4D-20-34-33-38-67-20-38-35-25-20-53-3A, "06:15 AM 438g 85% S:" received
- 18:16:07.221 Notification received from 0000fff1-0000-1000-8000-00805f9b34fb, value: (0x) 30-36-3A-31-35-20-41-4D-20-34-33-38-67-20-38-35-25-20-53-3A, "06:15 AM 438g 85% S:"
- 18:16:07.221 "(0x) 30-36-3A-31-35-20-41-4D-20-34-33-38-67-20-38-35-25-20-53-3A, "06:15 AM 438g 85% S:" received
- 18:16:37.279 Notification received from 0000fff1-0000-1000-8000-00805f9b34fb, value: (0x) 30-36-3A-31-36-20-41-4D-20-34-33-38-67-20-38-35-25-20-53-3A,

At the bottom, there is a 'DEBUG' button and a 'DEBUG' dropdown menu.